

Recurrent Neural Networks

RNNs for NLP Tasks

Introduction to Deep Learning · Class 29

Outline

The Recurrent Language Model

Labelling and Classification

Stacking and Direction

The RNN as a Probabilistic Model

Putting It Together

The Recurrent Language Model

The cell, put to work

- Class 27 built the encoder–decoder on a cell g ; Class 28 opened it. Today the cell is fixed and the question is **what to connect to it**
- Four output structures: a token at every step (**language modelling**), a label at every step (**sequence labelling**), one label for the whole sequence (**classification**), a new sequence (the **encoder–decoder**, Class 27)
- Two ways to combine cells: **stack** them, and run one **in each direction**
- Closing: the RNN as a probabilistic model, and its tree-shaped cousin

Jurafsky & Martin (2026), §14.2–14.4, §14.6; Goodfellow et al. (2016), §10.2.3, §10.5–10.6.

Notation for today

$\mathbf{x}_t \in \{0, 1\}^{ V }$	one-hot token, index w_t ; $\mathbf{e}_t = \mathbf{E}\mathbf{x}_t$ its embedding, $\mathbf{E} \in \mathbb{R}^{d \times V }$
d	the model dimension, shared by embeddings and states
$\mathbf{h}_t = g(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{e}_t)$	the cell of Classes 27–28 (simple, or an LSTM with $\mathbf{c}_t$ inside)
$\hat{\mathbf{y}}_t = \text{softmax}(\mathbf{V}\mathbf{h}_t)$	output distribution, $\mathbf{V} \in \mathbb{R}^{ V \times d}$; $\hat{y}_t[k]$ its k -th entry
$\mathbf{h}_t^f, \mathbf{h}_t^b, [\cdot; \cdot]$	forward and backward states; concatenation
$L_{\text{CE}}, \text{PPL}$	cross-entropy loss; perplexity
$y^{(t)}, h^{(t)}, \tau$	Goodfellow's superscripts and length, in the graphical-model slides only

Two conventions, one line

Jurafsky: $\mathbf{W}$ input, $\mathbf{U}$ recurrence, $\mathbf{V}$ output, time as a subscript. Goodfellow swaps $\mathbf{U}$ and $\mathbf{W}$ and writes $y^{(t)}$. This deck keeps Jurafsky's letters, as Classes 27–28 did.

Jurafsky & Martin (2026), §14.2.1; Goodfellow et al. (2016), §10.2.

The recurrent language model, recalled

Definition 1 (RNN language model)

At each step t ,

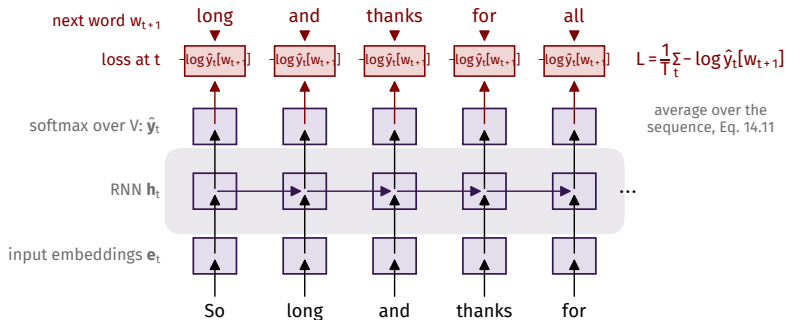
$$\mathbf{e}_t = \mathbf{E}\mathbf{x}_t, \quad \mathbf{h}_t = g(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{e}_t), \quad \hat{\mathbf{y}}_t = \text{softmax}(\mathbf{V}\mathbf{h}_t),$$

and $P(w_{t+1} = k \mid w_{1:t}) = \hat{y}_t[k]$. The probability of a sequence is the product of its next-token probabilities, $P(w_{1:n}) = \prod_{i=1}^n \hat{y}_i[w_i]$.

- Against the n -gram model, the context is not $n - 1$ tokens but **everything so far**, through $\mathbf{h}_{t-1}$; against the feedforward model, not a fixed window
- Shapes: $\mathbf{E}$ is $d \times |V|$, $\mathbf{U}, \mathbf{W}$ are $d \times d$, $\mathbf{V}$ is $|V| \times d$: the vocabulary enters only at the two ends

Jurafsky & Martin (2026), Eqs. 14.4–14.9, Fig. 14.5; Mikolov et al. (2010).

Training by self-supervision



At every position the model reads the true word, predicts the next, and pays the cross-entropy of the true next word,

$L_{\text{CE}}(\hat{y}_t, y_t) = -\sum_w y_t[w] \log \hat{y}_t[w] = -\log \hat{y}_t[w_{t+1}]$ for a one-hot target; the loss is the average over the sequence. No labels are added: the text is its own supervision. Teacher forcing: the input at $t + 1$ is the true w_{t+1} , not the prediction.

Layout of Jurafsky & Martin (2026), Fig. 14.6.

Jurafsky & Martin (2026), §14.2.2, Eqs. 14.10–14.11.

The training loss is the log of the perplexity

Proposition 2 (Perplexity and cross-entropy)

For a text $W = w_1 \dots w_N$, the perplexity $\text{PPL}(W) = P(w_1 \dots w_N)^{-1/N}$ equals exp of the mean per-token cross-entropy:

$$\text{PPL}(W) = \exp \left(\frac{1}{N} \sum_{t=1}^N -\log P(w_t \mid w_{<t}) \right).$$

- **Proof.** By the chain rule $P(w_{1:N}) = \prod_t P(w_t \mid w_{<t})$, so $-\frac{1}{N} \log P(w_{1:N}) = \frac{1}{N} \sum_t -\log P(w_t \mid w_{<t})$; exponentiate. □
- Reading: the **effective number of choices** per token — $|V|$ for a uniform model, 1 for a perfect one; per-token, so texts of different lengths compare. In bits, $\text{PPL} = 2^{H(W)}$.

Jurafsky & Martin (2026), Eqs. 3.14–3.15, 3.42, 7.58.

The training loss is the log of the perplexity

Proposition 2 (Perplexity and cross-entropy)

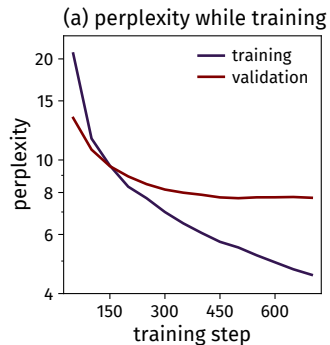
For a text $W = w_1 \dots w_N$, the perplexity $\text{PPL}(W) = P(w_1 \dots w_N)^{-1/N}$ equals exp of the mean per-token cross-entropy:

$$\text{PPL}(W) = \exp \left(\frac{1}{N} \sum_{t=1}^N -\log P(w_t \mid w_{<t}) \right).$$

- Proof. By the chain rule $P(w_{1:N}) = \prod_t P(w_t \mid w_{<t})$, so $-\frac{1}{N} \log P(w_{1:N}) = \frac{1}{N} \sum_t -\log P(w_t \mid w_{<t})$; exponentiate. □
- **Reading:** the effective number of choices per token — $|V|$ for a uniform model, 1 for a perfect one; per-token, so texts of different lengths compare. In bits, $\text{PPL} = 2^{H(W)}$.

Jurafsky & Martin (2026), Eqs. 3.14–3.15, 3.42, 7.58.

A language model, trained



(b) samples after training, prefix "Della "

temperature 0.5

Della and could had not of the sto she was she
stomed the soment and the looks, and his cruy no
whis look and be greare

temperature 1.0

Della adwade collant und haw it it thate quindouth,"
The contam olcoons thon wit im'll gotor or she nor
nerosich shell,

temperature 1.5

Della Rasmame1. Hef pwers, wapples pidiwat pretins
stAn let," neigerony thrllitsg lages dkown it pingled
lessom neslft: a

A character-level LSTM language model ($d = 64$, tied output) trained here on a public-domain story of 11,136 characters, the last 15% held out. (a) training perplexity keeps falling while validation perplexity flattens near 7.7 over 64 characters: the model starts to memorise. (b) samples at three temperatures: low is repetitive and English-like, high is diverse and wrong.

Jurafsky & Martin (2026), §14.2.2, §14.3.3, §7.6.3.

Generation: sampling, one token at a time

- **Autoregressive generation:** feed $\langle s \rangle$, sample a word from $\hat{y}_1$, feed its embedding back, sample again; stop at $\langle /s \rangle$ or a length limit. The text is a sample from $P(w_{1:n})$, not its mode
- **Temperature:** $\hat{y} = \text{softmax}(u/\tau)$; $\tau < 1$ sharpens towards the likely words, $\tau > 1$ flattens, $\tau \rightarrow 0$ is greedy decoding
- The same loop is Class 27's decoder, started from a context vector instead of $\langle s \rangle$: the priming makes it translation, summarisation or an answer

Class 27's greedy and beam decoding pick likely sequences; sampling picks typical ones. A language model is a distribution, and both are ways of reading it.

Jurafsky & Martin (2026), §14.3.3, Fig. 14.9, §7.6.2–7.6.3, Eq. 7.51; Shannon (1951).

Weight tying

Definition 3 (Tied language model)

The columns of $\mathbf{E}$ and the rows of $\mathbf{V}$ are both word vectors; a *tied* model uses one matrix for both:

$$\mathbf{e}_t = \mathbf{E} \mathbf{x}_t, \quad \mathbf{h}_t = g(\mathbf{U} \mathbf{h}_{t-1} + \mathbf{W} \mathbf{e}_t), \quad \hat{\mathbf{y}}_t = \text{softmax}(\mathbf{E}^\top \mathbf{h}_t).$$

Proposition 4 (What tying saves)

The embedding and output layers cost $|V| d$ instead of $2|V| d$: for $|V| = 50,000$, $d = 512$, that is 25.6M parameters fewer, with the recurrent core $2d^2$ unchanged.

- **Proof.** $\mathbf{E}$ and $\mathbf{V}$ have $|V|d$ entries each; the tied model keeps one. □
- The score of word k is $\mathbf{e}_k^\top \mathbf{h}_t$: the next word is the one whose embedding the state points at.

Jurafsky & Martin (2026), §14.2.3, Eqs. 14.12–14.14; Press & Wolf (2017); Inan et al. (2017).

Weight tying

Definition 3 (Tied language model)

*The columns of $\mathbf{E}$ and the rows of $\mathbf{V}$ are both word vectors; a **tied** model uses one matrix for both:*

$$\mathbf{e}_t = \mathbf{E} \mathbf{x}_t, \quad \mathbf{h}_t = g(\mathbf{U} \mathbf{h}_{t-1} + \mathbf{W} \mathbf{e}_t), \quad \hat{\mathbf{y}}_t = \text{softmax}(\mathbf{E}^\top \mathbf{h}_t).$$

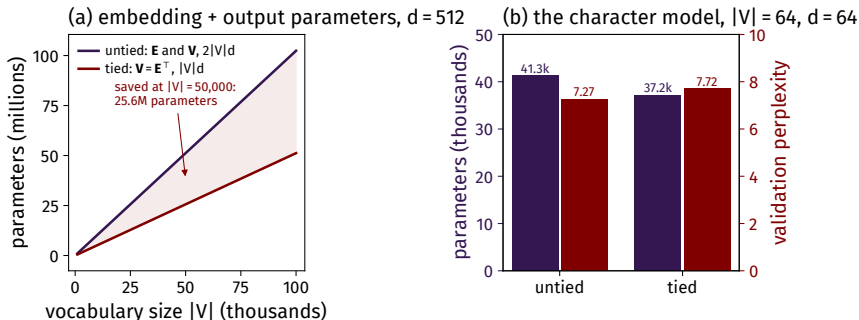
Proposition 4 (What tying saves)

The embedding and output layers cost $|V|d$ instead of $2|V|d$: for $|V| = 50,000$, $d = 512$, that is 25.6M parameters fewer, with the recurrent core $2d^2$ unchanged.

- Proof. $\mathbf{E}$ and $\mathbf{V}$ have $|V|d$ entries each; the tied model keeps one. □
- **The score of word k is $\mathbf{e}_k^\top \mathbf{h}_t$: the next word is the one whose embedding the state points at.**

Jurafsky & Martin (2026), §14.2.3, Eqs. 14.12–14.14; Press & Wolf (2017); Inan et al. (2017).

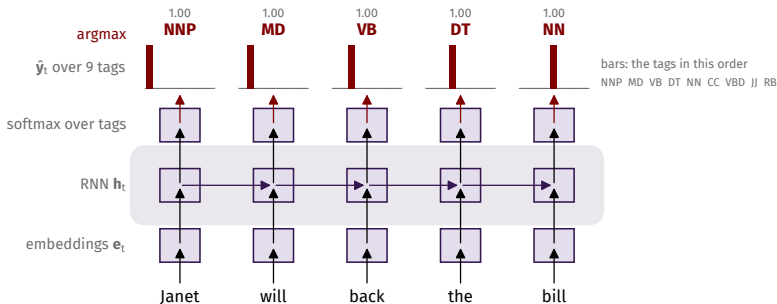
Weight tying, counted and tried



(a) embedding and output parameters against vocabulary size at $d = 512$: tying halves them. (b) the character model trained here both ways: at $|V| = 64$ the saving is 4k of 41k parameters, and on this small text the untied model reaches a slightly lower validation perplexity (7.27 against 7.72). The saving scales with $|V|$; the perplexity gain the book reports is for word vocabularies. Jurafsky & Martin (2026), §14.2.3.

Labelling and Classification

Sequence labelling: a label at every step



Part-of-speech tagging as sequence labelling: embeddings in, an RNN, and a softmax over the tag set at every step; the tag is the argmax — the language model's output layer with the tag set in place of the vocabulary, the cross-entropy summed over the steps, and a local decision at each. The distributions shown are those of the left-to-right tagger trained here on a small grammar with 9 tags, for the book's sentence. Layout of Jurafsky & Martin (2026), Fig. 14.7; §14.3.1.

Sequence classification: one label per sequence

Definition 5 (Read-outs for classification)

Run the RNN over $x_1 \dots x_n$ and pass one vector to a feedforward classifier with a softmax over the classes; three choices of the vector:

$$\mathbf{h}_n \text{ (the last state),} \quad \mathbf{h}_{\text{mean}} = \frac{1}{n} \sum_{i=1}^n \mathbf{h}_i, \quad \mathbf{h}_{\text{max}} = \max_i \mathbf{h}_i \text{ (elementwise).}$$

- No loss at earlier positions: the one error signal goes back through the classifier, then the RNN — **end-to-end training**. $\mathbf{h}_n$ must carry the whole text; pooling reads every state

Jurafsky & Martin (2026), §14.3.2, Eq. 14.15, Fig. 14.8.

Why the read-out matters

Proposition 6 (Gradient paths of the three read-outs)

Let L be the classification loss. For the last-state read-out the gradient reaches $\mathbf{h}_t$ only through $n - t$ recurrent steps, $\partial L / \partial \mathbf{h}_t = (\partial L / \partial \mathbf{h}_n) \prod_{s=t+1}^n \mathbf{J}_s$; for the mean it also arrives directly, with weight $1/n$; for the max, with weight 1 on every coordinate where $\mathbf{h}_t$ attains the max.

- **Proof.** Chain rule along the unrolled graph: the last state has one path back, the recurrence; the pooled read-outs add an edge from every $\mathbf{h}_t$ to the read-out, whose derivative is $\frac{1}{n} \mathbf{l}$ or a 0/1 mask. □
- By Class 28, the product $\prod \mathbf{J}_s$ shrinks geometrically for a simple cell; the direct edge does not. If the evidence is at the start of a long text, the last-state classifier may never see it during training.

Jurafsky & Martin (2026), §14.3.2, in words; Class 28, Proposition 2.

Why the read-out matters

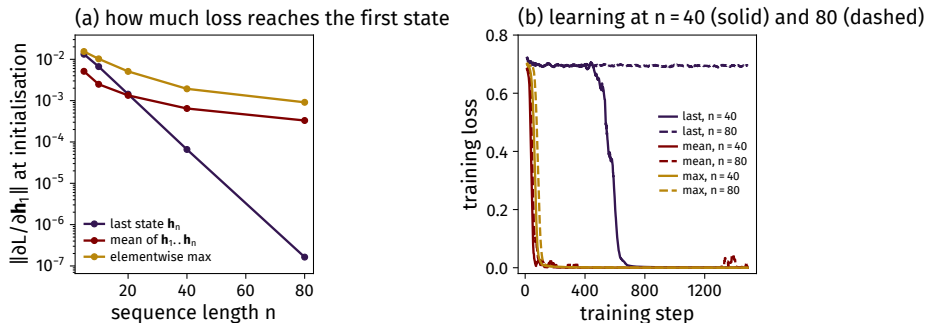
Proposition 6 (Gradient paths of the three read-outs)

Let L be the classification loss. For the last-state read-out the gradient reaches $\mathbf{h}_t$ only through $n - t$ recurrent steps, $\partial L / \partial \mathbf{h}_t = (\partial L / \partial \mathbf{h}_n) \prod_{s=t+1}^n \mathbf{J}_s$; for the mean it also arrives directly, with weight $1/n$; for the max, with weight 1 on every coordinate where $\mathbf{h}_t$ attains the max.

- Proof. Chain rule along the unrolled graph: the last state has one path back, the recurrence; the pooled read-outs add an edge from every $\mathbf{h}_t$ to the read-out, whose derivative is $\frac{1}{n} \mathbf{I}$ or a 0/1 mask. □
- By Class 28, the product $\prod \mathbf{J}_s$ shrinks geometrically for a simple cell; the direct edge does not. If the evidence is at the start of a long text, the last-state classifier may never see it during training.

Jurafsky & Martin (2026), §14.3.2, in words; Class 28, Proposition 2.

The three read-outs, measured



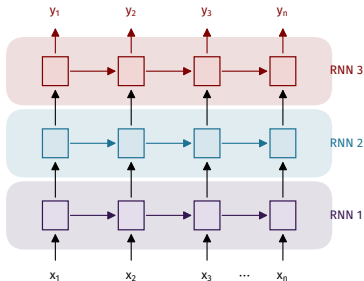
A simple-RNN classifier ($d = 16$) on sequences whose label is decided by one marker among the first three symbols, the rest filler. (a) the gradient reaching h_1 at initialisation, read off the tape: through the last state it falls by 10^{-5} from $n = 5$ to 80; through the mean and the max it barely moves. (b) training at fixed length: at $n = 40$ the last-state model sits at chance for 500 steps before it learns; at $n = 80$ it never does, while both pooled models learn within 100 steps.

Jurafsky & Martin (2026), §14.3.2.

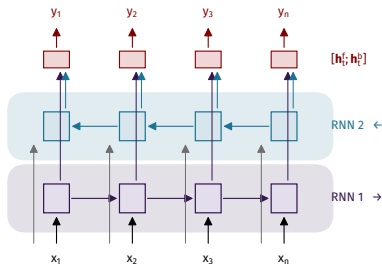
Stacking and Direction

RNNs as modules

(a) stacked: three layers



(b) bidirectional: states concatenated, Eq. 14.18



Unrolled, an RNN is a feedforward graph with vectors in and out, so it composes like any other layer. (a) stacked: the output sequence of one layer is the input sequence of the next; the top layer's outputs are the final outputs. (b) bidirectional: one network reads left to right, one right to left, and their states at each step are concatenated. Layouts of Jurafsky & Martin (2026), Figs. 14.10–14.11; §14.4.

Stacking: depth per time step

Proposition 7 (What stacking buys)

L layers of width d cost $O(Ld^2)$ recurrent parameters, the same as one layer of width $\sqrt{L}d$. At equal parameter count the stacked network is deeper between $\mathbf{x}_t$ and $\hat{\mathbf{y}}_t$ (L nonlinearities instead of one) and no deeper across time.

- **Proof.** Each layer has $\mathbf{U}, \mathbf{W} \in \mathbb{R}^{d \times d}$, so $2d^2$ per layer; one layer of width $d' = \sqrt{L}d$ has $2Ld^2$. The path from $\mathbf{h}_t^{(\ell)}$ to $\mathbf{h}_{t+1}^{(\ell)}$ is still one step. □
- Goodfellow's three ways to make an RNN deep: (a) stack the state hierarchically; (b) put an MLP in the input-to-hidden, hidden-to-hidden or hidden-to-output map — which lengthens the shortest path across time; (c) add skip connections to shorten it again.
- Jurafsky: lower layers induce representations the upper layers use; the right depth is task-specific.

Jurafsky & Martin (2026), §14.4.1; Goodfellow et al. (2016), §10.5, Fig. 10.13; Graves et al. (2013); Pascanu et al. (2014).

Stacking: depth per time step

Proposition 7 (What stacking buys)

L layers of width d cost $O(Ld^2)$ recurrent parameters, the same as one layer of width $\sqrt{L}d$. At equal parameter count the stacked network is deeper between $\mathbf{x}_t$ and $\hat{\mathbf{y}}_t$ (L nonlinearities instead of one) and no deeper across time.

- Proof. Each layer has $\mathbf{U}, \mathbf{W} \in \mathbb{R}^{d \times d}$, so $2d^2$ per layer; one layer of width $d' = \sqrt{L}d$ has $2Ld^2$. The path from $\mathbf{h}_t^{(\ell)}$ to $\mathbf{h}_{t+1}^{(\ell)}$ is still one step. $\square$
- **Goodfellow's three ways to make an RNN deep:** (a) stack the state hierarchically; (b) put an MLP in the input-to-hidden, hidden-to-hidden or hidden-to-output map — which lengthens the shortest path across time; (c) add skip connections to shorten it again.
- Jurafsky: lower layers induce representations the upper layers use; the right depth is task-specific.

Jurafsky & Martin (2026), §14.4.1; Goodfellow et al. (2016), §10.5, Fig. 10.13; Graves et al. (2013); Pascanu et al. (2014).

Stacking: depth per time step

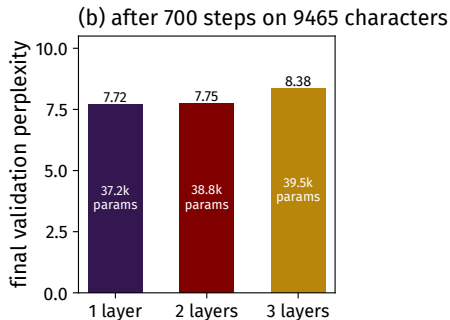
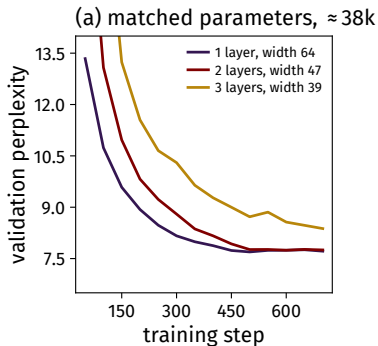
Proposition 7 (What stacking buys)

L layers of width d cost $O(Ld^2)$ recurrent parameters, the same as one layer of width $\sqrt{L}d$. At equal parameter count the stacked network is deeper between $\mathbf{x}_t$ and $\hat{\mathbf{y}}_t$ (L nonlinearities instead of one) and no deeper across time.

- Proof. Each layer has $\mathbf{U}, \mathbf{W} \in \mathbb{R}^{d \times d}$, so $2d^2$ per layer; one layer of width $d' = \sqrt{L}d$ has $2Ld^2$. The path from $\mathbf{h}_t^{(\ell)}$ to $\mathbf{h}_{t+1}^{(\ell)}$ is still one step. $\square$
- Goodfellow's three ways to make an RNN deep: (a) stack the state hierarchically; (b) put an MLP in the input-to-hidden, hidden-to-hidden or hidden-to-output map — which lengthens the shortest path across time; (c) add skip connections to shorten it again.
- **Jurafsky: lower layers induce representations the upper layers use; the right depth is task-specific.**

Jurafsky & Martin (2026), §14.4.1; Goodfellow et al. (2016), §10.5, Fig. 10.13; Graves et al. (2013); Pascanu et al. (2014).

Stacking, tried at matched parameters



One, two and three LSTM layers, widths 64, 47, 39, each about 38k parameters, trained the same way on the character task. (a) validation perplexity by step; (b) at the end, 7.72, 7.75, 8.38: on 9,465 characters depth does not pay, and three layers are slower to train. The gain the books report is for data that has levels of abstraction to find.

Jurafsky & Martin (2026), §14.4.1; Goodfellow et al. (2016), §10.5.

Bidirectionality: the right context

Definition 8 (Bidirectional RNN, as used)

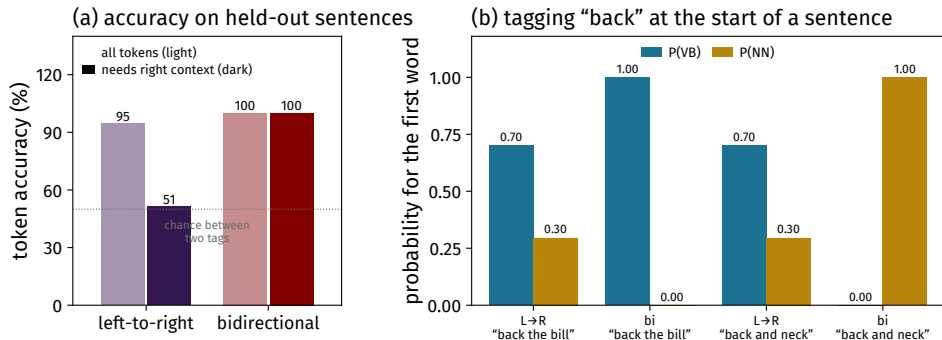
Two independent RNNs, concatenated at each step:

$$\mathbf{h}_t^f = \text{RNN}_{\text{forward}}(x_1, \dots, x_t), \quad \mathbf{h}_t^b = \text{RNN}_{\text{backward}}(x_t, \dots, x_n), \quad \mathbf{h}_t = [\mathbf{h}_t^f; \mathbf{h}_t^b].$$

- $\mathbf{h}_t^f$ is the ordinary state, everything to the left; $\mathbf{h}_t^b$ the state of a network run on the reversed input, everything to the right
- **Labelling**: the decision at t sees both sides. **Classification**: concatenate the two final states, so the beginning counts as much as the end. Needs the whole input, so not for generation

Jurafsky & Martin (2026), §14.4.2, Eqs. 14.16–14.18, Figs. 14.11–14.12; Schuster & Paliwal (1997); Class 26.

Bidirectionality, measured



Two taggers trained here on a grammar in which a word such as *back* opens either an imperative (*back the bill*: VB) or a noun phrase (*back and neck ached*: NN). (a) on held-out sentences the left-to-right tagger scores 95% overall but 51% on those sentence-initial words — chance between two tags, since at $t = 1$ it has seen nothing else; the bidirectional tagger scores 100%. (b) its distributions for *back*: the same 0.7/0.3 in both sentences for the left-to-right model, and the right answer for the bidirectional one.

Jurafsky & Martin (2026), §14.4.2.

The RNN as a Probabilistic Model

The RNN as a directed graphical model

Proposition 9 (Parameter sharing over the complete graph)

An RNN over $y^{(1)} \dots y^{(\tau)}$ with no other input defines $P(\mathbf{Y}) = \prod_{t=1}^{\tau} P(y^{(t)} | y^{(t-1)}, \dots, y^{(1)})$, with $L = \sum_t L^{(t)}$ and $L^{(t)} = -\log P(y^{(t)} | y^{(<t)})$: the complete graph over the y 's. A tabular parametrisation with k -valued y needs $O(k^{\tau})$ parameters; the RNN's count is $O(1)$ in τ .

- **Proof.** The chain rule of probability gives the product; every conditional has the whole past as parents. The table for $P(y^{(\tau)} | y^{(<\tau)})$ alone has k^{τ} entries; the RNN reuses f and θ at every step. □
- The state makes it possible: every stage has the same structure and parameters; a distant $y^{(i)}$ reaches $y^{(t)}$ through $h^{(t)}$.

Goodfellow et al. (2016), §10.2.3, Eqs. 10.29–10.33.

The RNN as a directed graphical model

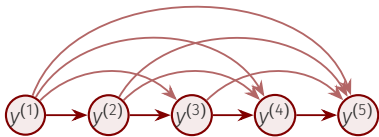
Proposition 9 (Parameter sharing over the complete graph)

An RNN over $y^{(1)} \dots y^{(\tau)}$ with no other input defines $P(\mathbf{Y}) = \prod_{t=1}^{\tau} P(y^{(t)} | y^{(t-1)}, \dots, y^{(1)})$, with $L = \sum_t L^{(t)}$ and $L^{(t)} = -\log P(y^{(t)} | y^{(<t)})$: the complete graph over the y 's. A tabular parametrisation with k -valued y needs $O(k^{\tau})$ parameters; the RNN's count is $O(1)$ in τ .

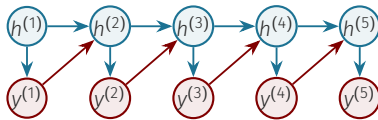
- Proof. The chain rule of probability gives the product; every conditional has the whole past as parents. The table for $P(y^{(\tau)} | y^{(<\tau)})$ alone has k^{τ} entries; the RNN reuses f and θ at every step. □
- **The state makes it possible: every stage has the same structure and parameters; a distant $y^{(i)}$ reaches $y^{(t)}$ through $h^{(t)}$.**

Goodfellow et al. (2016), §10.2.3, Eqs. 10.29–10.33.

The two graphs, drawn



(a) the complete graph: every past $y^{(i)}$ is a parent of $y^{(t)}$



(b) with the state: the same structure and parameters at every stage

- Marginalise the states out of (b) and (a) remains; keep them and $y^{(t)}$ reaches everything before through $h^{(t)}$
- The price: **stationarity**, the same conditional at every t ; and some operations stay hard, such as filling a gap in the middle

Goodfellow et al. (2016), Figs. 10.7–10.8, §10.2.3.

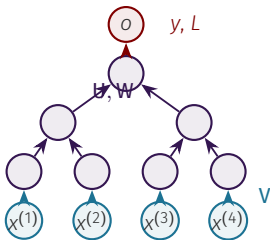
Deciding when to stop

- A model over sequences must also decide their **length**. Three mechanisms, all trained from data:
- An **end symbol** $\langle /s \rangle$ in the vocabulary, inserted after the last token of every training sequence; sampling stops when it is drawn. This is Class 27's decoder and today's language model
- A **Bernoulli output** at every step, a sigmoid trained by cross-entropy to say “continue” or “halt”; works for any output type, real-valued sequences included
- **Predict τ first**, then sample τ steps, feeding τ or $\tau - t$ back as an extra input so the model knows how close the end is:

$$P(x^{(1)}, \dots, x^{(\tau)}) = P(\tau) P(x^{(1)}, \dots, x^{(\tau)} \mid \tau)$$

Goodfellow et al. (2016), §10.2.3, Eq. 10.34; Schmidhuber (2012).

The tree-shaped cousin: recursive networks



A recursive network: a chain becomes a tree, with one set of weights at every node. Layout of Goodfellow et al. (2016), Fig. 10.14.

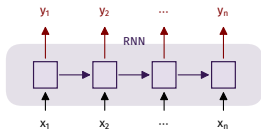
- The same idea as the RNN — one function, one parameter set, applied repeatedly — on a **tree** instead of a chain; a variable-length sequence still maps to one vector
- Depth: the number of nonlinear compositions from a leaf to the root is $O(\log \tau)$ for a balanced tree, against τ for the chain, which helps with long dependencies
- The open question is the tree: fixed and balanced, given by a parser (the sentence's parse tree, Socher et al.), or inferred by the learner

Goodfellow et al. (2016), §10.6, Fig. 10.14; Pollack (1990); Socher et al. (2011, 2013).

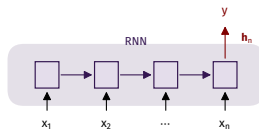
Putting It Together

Four architectures

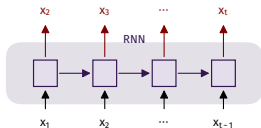
(a) sequence labelling



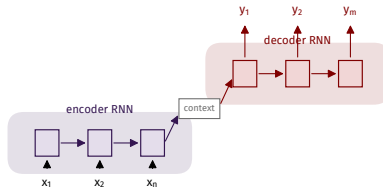
(b) sequence classification



(c) language modelling



(d) encoder-decoder



(a) sequence labelling: each x_i to a y_i . (b) classification: the whole sequence to one class, through the last state or a pooled one. (c) language modelling: the next token at every step. (d) encoder-decoder: one RNN maps the input to a context, another maps the context to an output sequence of its own length.

Layout of Jurafsky & Martin (2026), Fig. 14.15.

The block, in one table

	output	loss	where the gradient enters
language model	$\hat{y}_t$ over V , every step	$-\log \hat{y}_t[w_{t+1}]$, averaged	every step
sequence labelling	$\hat{y}_t$ over tags, every step	cross-entropy, every step	every step
classification	one softmax	one cross-entropy	$\mathbf{h}_n$, or every $\mathbf{h}_t$ if pooled
encoder-decoder (27)	$\hat{y}_t$ over V , decoder steps	teacher-forced cross-entropy	decoder steps, back through $\mathbf{c}$
stacked	top layer's outputs	as above	through L cells per step
bidirectional	$[\mathbf{h}_t^f; \mathbf{h}_t^b]$	as above	from both directions

One cell, three matrices, and the choice of what to read out and where to pay the loss: that choice is the architecture.

Takeaways

1. A language model is an RNN whose output is the next token; it trains on raw text by self-supervision, and its loss is the log of its perplexity (Definition 1, Proposition 2)
2. Tying the output matrix to the embeddings removes $|V|d$ parameters (Definition 3, Proposition 4); sampling with a temperature reads the model as a distribution
3. Labelling emits a distribution at every step; classification reads out one vector, and pooling gives the early tokens a direct path to the loss (Definition 5, Proposition 6) — measured: at $n = 80$ the last-state classifier never learns, the pooled ones do
4. Stacking buys depth per step at the same parameter count (Proposition 7); bidirectionality buys the right context (Definition 8) — measured: 51% $\rightarrow$ 100% on the tokens that need it
5. The RNN is the complete graphical model over the sequence with $O(1)$ parameters in its length (Proposition 9); the recursive net is the same idea on a tree

Classes 30–31: the recurrence goes, attention stays — the transformer.

References i

Questions?